Turbine Judgement Gym — Hackathon Build Plan

Oct 31, 2026 · @Liana

Everything in one picture

We build a Track B judgement gym: a junior replays real turbine faults, commits a call, gets challenged by a Gemini senior, and is scored by code against what really happened.



System overview · data, app, logic, mentor · owners P1–P5

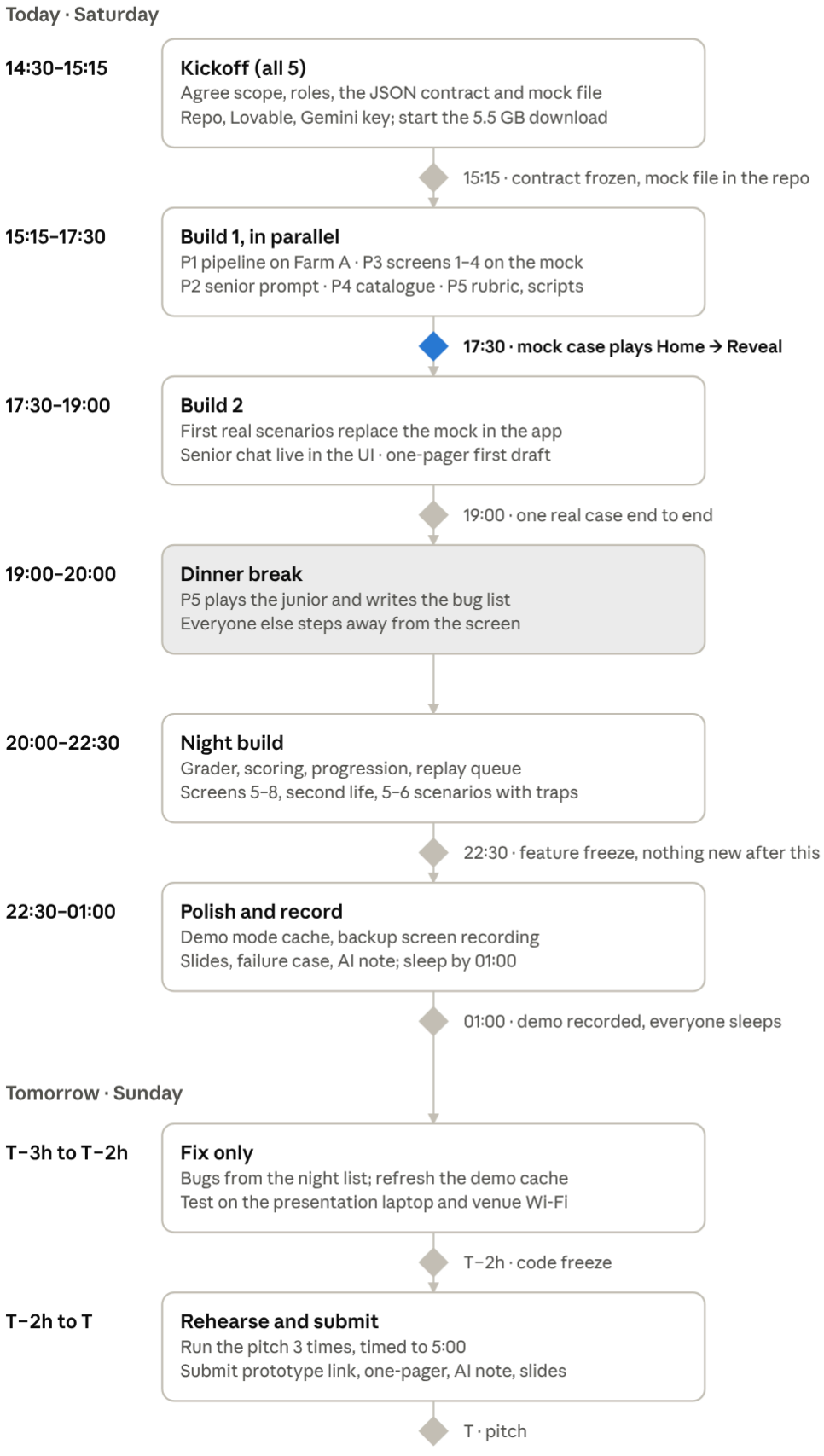
Read it left to right. P1 turns raw SCADA data into cases, P3′s app walks the junior through seven steps, and P2′s logic challenges, scores and picks the next case. A human mentor reads the result.

Timeline

The whole plan hangs on 17:30: by then the mock case must play from Home to Reveal, even if ugly. After that, real data and features only get added to something that already works.

One end-to-end case by 17:30, feature freeze at 22:30

Planned times, not to scale · T = your presentation time on Sunday



Build timeline · 8 phases, 7 gates

If a gate slips by more than 30 minutes, cut scope from the Risks table instead of pushing the gate. Per-person tasks for each block are in the second Team split table.

Team split

Five owners, one shared contract: the scenario JSON file (see Data). From 15:15 everyone builds against a mock copy of that file, so nobody waits for anyone else.

Role	Best fit	Owns	Hands to
P1 Data lead	Strongest Python / numerics person	Dataset download, preprocessing, anomaly model, scenario JSON files, traps planted in the data, cost numbers	Scenario files to P3; hidden truth labels to P2
P2 Sim lead	Likes prompts and backend logic	Gemini calls (senior, grader, reuse agent), server functions holding the key, scoring, progression, replay queue	Working API functions to P3
P3 UI lead	Most comfortable with Lovable / React	All screens, charts, local storage of the junior's log, demo mode	The running app to everyone
P4 Second-life lead	Architecture PhD	Reuse and recycling catalogue with real links, the planted fake option, end-of-life cost and CO2 assumptions, reuse agent prompt (with P2)	Catalogue JSON to P2 and P3
P5 Learning and story lead	Architecture PhD, or whoever pitches best	Skill map, grading rubric, scenario scripts (briefing, copilot suggestion, senior questions), one-pager, pitch, AI usage note; plays the junior in every test	Rubric to P2; scripts to P1

Role	Today 14:30–19:00	Night 20:00–01:00	Tomorrow (2–3 h)
P1 Data	Download and load Farm A, pick events, build the anomaly model, first real scenario by 17:30, 5 scenarios by 19:00	Plant traps, add costs, build the end-of-life case and 1 replay variant, check every JSON loads in the app	On call for data questions; one backup chart for the slides
P2 Sim	AI Studio key, server function, senior chat working on the mock by 17:30	Grader with JSON output, reuse agent with search, scoring, progression and replay queue, demo-mode cache	Bug fixes only; refresh the demo cache
P3 UI	Lovable project linked to GitHub; screens 1–4 on the mock	Real JSON and Gemini functions wired in, screens 5–8, polish	Code freeze; run the demo twice
P4 Second-life	Catalogue of 8–12 real reuse / recycling routes; the fake option; component end-of-life table	Second-life screen content, cost and CO2 numbers, check reuse agent output	Circularity slide; answer judge questions
P5 Story	Skill map and rubric by 15:15; scenario scripts; one-pager draft	Play the junior on every build, write the failure case, slides, AI usage note	Rehearse the pitch 3 times; submit everything

Data

We use one public dataset, CARE to Compare (Fraunhofer IEE, CC BY-SA 4.0), Wind Farm A only. Python preprocesses it offline into 5–6 small JSON scenario files, and the app only ever reads those files. The LLM never touches raw CSVs.

Sources

Source	What we take	Notes
CARE to Compare on Zenodo	Farm A: 5 onshore turbines in Portugal, 86 features, labelled fault events	5.5 GB zip. Avoid version 2 (broken timestamps). Take the newest version, which has corrected labels. Start the download now.
ENTSO-E Transparency Platform	Day-ahead price for Portugal, a few typical weeks	Turns downtime into euros. A flat 70 €/MWh is an acceptable fallback, labelled as an assumption.
P4's reuse catalogue (hand-made)	8–12 real reuse / recycling routes with links	Re-Wind design catalogue, Blade Made, BladeBridge, cement co-processing, steel and copper recycling
Synthetic (ours)	Repair costs, crew availability, weather window, turbine age	Every invented number gets an "illustrative" label in the UI

Pipeline, step by step (P1)

1. Unzip only the Farm A folder. Read the README, the event info file and the feature description file first. They tell you which dataset is a fault and which is normal, the event start and end, which column is which sensor, and which rows are training vs prediction period. Check the real column names there; don't guess.
2. Pick the cases: 3–4 fault events with clear temperature-type signals (gearbox, generator bearing, transformer, hydraulics), plus 2 normal datasets to use as false-alarm cases.
3. For each case keep about 10 signals: wind speed, power, rotor speed, ambient temperature, nacelle temperature, and the temperatures of the affected component. Use only the Avg columns; the dataset authors warn that Min, Max and Std are often implausible.
4. Build a normal behaviour model per turbine. Train a regression on the training period, normal status rows only: target = component temperature, inputs = power, wind speed, ambient temperature, rotor speed. Start with Ridge regression; try gradient boosting only if time allows.
5. Compute the residual and a threshold, then flag the turbine when the 24-hour rolling mean of the residual stays above the threshold.

6. Cut the scenario window: about 21 days of history up to a "decision date" a couple of weeks before the logged event. Everything after the decision date is the hidden future, shown only at the reveal. Resample to hourly (about 500 points per signal) so the charts stay fast.
7. Plant traps (see Simulation engine): a flat-lined sensor, a spike caused by a heatwave rather than a fault, or a wrong copilot suggestion. Write each trap into the JSON so scoring knows it is there.
8. Add costs per action and downtime per day: estimated lost MWh × price.
9. Export each case to `public/scenarios/<id>.json` and run a check script that every file matches the contract.

```
r_t = y_t - \hat{y}_t, \qquad \text{alarm when } \overline{r}_{24h} >
\mu_{r,\text{train}} + 3\,\sigma_{r,\text{train}}
```

Here y is the measured component temperature and ŷ the model's prediction for the same conditions. A healthy turbine has a residual near zero; a failing bearing runs hotter than predicted, often for days or weeks before failure.

The scenario contract. P1 writes the mock version of this file by 15:15. P2 and P3 build against it.

```
{
  "id": "A-gearbox-01",
  "variant_of": null,
  "difficulty": 1,
  "skills": ["read_signal", "challenge_ai", "weigh_cost"],
  "turbine": { "farm": "A", "asset": "T3", "age_years": 14 },
  "briefing": {
    "decision_date": "2017-06-12",
    "text": "Monday morning. One crew is free on Thursday. Storm forecast for
the weekend."
  },
  "series": {
    "t": ["2017-05-22T00:00", "..."],
    "signals": {
      "gearbox_bearing_temp": { "unit": "°C", "actual": [], "expected": [] },
      "power": { "unit": "kW", "actual": [] },
      "wind_speed": { "unit": "m/s", "actual": [] }
    },
    "residual": { "values": [], "threshold": 3.1 }
  },
  "alarms": [{ "t": "2017-06-09T14:00", "text": "Gearbox bearing temp high
(warning)" }],
  "copilot": { "text": "Pattern looks like sensor drift. Keep monitoring.",
```

```
"confidence": 0.72, "is_wrong": true },
  "traps": [{ "type": "wrong_ai", "explanation": "Neighbouring sensors rise
too; drift would affect one sensor only." }],
  "actions": ["monitor", "inspect", "repair", "replace_component",
"decommission"],
  "truth": {
    "label": "fault",
    "component": "gearbox bearing",
    "days_to_failure": 18,
    "best_action": "inspect",
    "acceptable": ["inspect", "repair"]
  },
  "future": { "t": [], "gearbox_bearing_temp": [] },
  "costs_eur": { "monitor": 0, "inspect": 2500, "repair": 40000,
"replace_component": 250000, "downtime_per_day": 1900 },
  "end_of_life": { "eligible": false }
}
```

Fields under `truth`, `future` and `traps` are never shown before the reveal and never sent to the junior's view. The dates and costs above are placeholders; P1 fills real ones.

Simulation engine with Gemini

Gemini plays three roles: the sceptical senior, the reasoning grader and the second-life research agent. Anything that decides a score (was the decision right, was the trap caught) is plain code against the hidden truth in the JSON. That keeps traps controlled and scores reproducible, and it is our main oversight argument to the judges.

One case, start to finish

1. **Briefing.** Turbine, date, time pressure, crew availability. No AI.
2. **Investigate.** The junior explores charts, alarms and a data-quality panel. The copilot suggestion card comes from the JSON, not live Gemini, so a planted wrong suggestion is guaranteed. The app logs every check the junior opens.
3. **Think first.** The junior commits before talking to the senior: action, confidence (0–100 %), reason, "what would change my mind", and two flags: "I suspect a data problem in signal X" and "I disagree with the copilot".
4. **Senior challenge.** Gemini asks up to 3 questions, one at a time. The junior answers and may revise the decision once. Every revision is logged.
5. **Reveal.** The app shows the hidden future data and the real outcome, the cost of each action in euros, and whether a trap was present and caught.
6. **Grade.** Code computes decision, trap, checks and calibration points. Gemini grades the written reasoning against the rubric and returns JSON.

7. **Second life (only if the action is replace or decommission).** The reuse agent researches options; the junior marks each one verified, unverified or rejected. One option is planted and fake.
8. **Update.** Skill levels change, a missed trap goes into the replay queue, and the judgement profile updates.

The three Gemini roles

Role	Called at	Gets	Returns	Settings
Senior	Step 4, each turn	Case data the junior saw, the committed decision, the chat so far, a list of checks the junior skipped (computed by code), help level	One short question, or `READY_FOR_REVEAL`	Temperature 0.6, plain text, multi-turn
Grader	Step 6, once	Decision, reasoning text, senior chat, the truth	JSON: 4 rubric scores, short feedback, what to practise next	Temperature 0, JSON schema
Reuse agent	Step 7, once	Component, material, turbine age, region	JSON list of options with a source link each	Google Search grounding on

The senior does **not** get the truth label. It gets only what the junior skipped, so it can push on blind spots without leaking the answer. Leakage is the most likely way an LLM ruins the gym.

Senior system prompt (first draft, P2 and P5 refine it)

```
You are Dr. Keller, a senior asset manager with 20 years in wind operations.
You are demanding but fair. A junior colleague has just committed a decision
on a turbine case. Challenge the reasoning, never just the answer.

Rules:
- Ask ONE question per turn, at most two sentences. No lists. No praise.
- Never say whether the decision is right or what will happen next.
- Pick from these question types, aimed at the weakest part of the reasoning:
  what would change your mind / what is the downside if you are wrong /
  what did you not check / can you trust this data / what does waiting cost.
- SKIPPED_CHECKS lists what the junior never looked at. Use it.
- HELP_LEVEL {0-3}: at 0 you may add one hint sentence; at 2 or 3 never hint.
- After the junior's third answer, or when they write "final", reply exactly:
  READY_FOR_REVEAL

CASE: {briefing, key signal summary, alarms, copilot text}
DECISION: {action, confidence, reason, mind_changer, flags}
SKIPPED_CHECKS: {e.g. "never opened generator bearing 2", "ignored storm
forecast"}
```

Grader output schema (sent as `responseSchema`, so the answer always parses)

```
{
  "type": "object",
  "properties": {
    "evidence":     { "type": "integer", "description": "0-3: cites specific
signals and numbers" },
    "alternatives": { "type": "integer", "description": "0-3: considered
other explanations (sensor, weather, curtailment)" },
    "downside":     { "type": "integer", "description": "0-3: named the cost
of being wrong" },
    "mind_changer": { "type": "integer", "description": "0-3: named a
concrete observation that would change the call" },
    "feedback":     { "type": "string" },
    "practise_next":{ "type": "string" }
  },
  "required": ["evidence", "alternatives", "downside", "mind_changer",
"feedback", "practise_next"]
}
```

Reuse agent prompt (P4 and P2)

```
You research second-life options for a decommissioned wind turbine component.
COMPONENT: {e.g. 3 x 40 m glass-fibre blades, 14 years old} REGION: {Portugal
/ Switzerland}
Find 4-6 real options: reuse in architecture or infrastructure,
remanufacture,
resale as spare part, recycling route. For each give: name, type, who runs
it,
country, rough value or cost if published, and the source URL you used.
If you cannot find a source, say "no source". Never invent a project.
Return only a JSON array.
```

The app then merges the agent's options with P4's catalogue and inserts one planted fake option, for example an architecture project that doesn't exist, or one that needs a blade length this turbine doesn't have. The junior is scored on whether they catch it. With Search grounding on, JSON schema mode may not be allowed in the same call; if so, ask for JSON in the text and parse it with a try/catch.

The one server function that holds the key

Never put the Gemini key in browser code or in the repo. In Lovable, enable Cloud / Supabase, add `GEMINI_API_KEY` as a secret, and route all three roles through one function:

```
// supabase/functions/gemini/index.ts
const KEY = Deno.env.get("GEMINI_API_KEY")!;
const MODEL = Deno.env.get("GEMINI_MODEL") ?? "gemini-2.5-flash"; // use the
newest Flash your key lists
const cors = { "Access-Control-Allow-Origin": "*", "Access-Control-Allow-
Headers": "*" };

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return new Response(null, { headers: cors });
  const { system, messages, schema, search, temperature = 0.4 } = await
req.json();
  const body = {
    systemInstruction: { parts: [{ text: system }] },
    contents: messages.map((m) => ({ role: m.role, parts: [{ text: m.text }]
})), // role: "user" | "model"
    generationConfig: { temperature, ...(schema ? { responseMimeType:
"application/json", responseSchema: schema } : {}) },
    ...(search ? { tools: [{ google_search: {} }] } : {}),
  };
  const r = await fetch(

`https://generativelanguage.googleapis.com/v1beta/models/${MODEL}:generateCon
tent`,
```

```
    { method: "POST", headers: { "Content-Type": "application/json", "x-goog-
api-key": KEY }, body: JSON.stringify(body) },
  );
  const j = await r.json();
  const text = j.candidates?.[0]?.content?.parts?.map((p) => p.text ??
"").join("") ?? "";
  return new Response(JSON.stringify({ text, grounding: j.candidates?.
[0]?.groundingMetadata ?? null }),
    { headers: { ...cors, "Content-Type": "application/json" } });
});
```

To see which model names your key can use, call `GET https://generativelanguage.googleapis.com/v1beta/models` with the same header and pick the newest Flash model; Flash is fast enough for live chat.

Scoring, 100 points per case (code, `scoring.ts`)

Part	Rule	Points
Decision	Best action 40, acceptable 25, otherwise 0	40
Trap	Wrong copilot: "disagree" flag set and action differs from the copilot's. Data glitch: the glitched signal is flagged. False alarm: no repair or replace on a healthy turbine.	25
Checks	Opened the residual, a corroborating sensor and the data-quality panel	10
Reasoning	Gemini rubric, 4 criteria × 0–3, scaled to 15	15
Calibration	$10 \times (1 - (\text{confidence} - \text{correct})^2)$, with correct = 1 or 0	10

The euro outcome (cost of the chosen action plus downtime) is shown next to the score but not added to it, so the junior sees the business result separately from the quality of their judgement.

Skill map and progression (`progression.ts`)

Skill	Shown by
Read the signal	Residual drift told apart from normal variation
Check the data	Flat-lined or implausible sensors caught
Challenge the AI	Wrong copilot suggestions overruled, with a reason

Skill	Shown by
Weigh cost and risk	Action fits downtime cost, crew slot and weather window
Second-life judgement	Fake or unsourced reuse options caught; a sound end-of-life route chosen

- Each skill has a level from 0 to 3. A success on a skill the case tests moves it up 1; a failure moves it down 1.
- The next case targets the lowest skill, at difficulty = that level + 1.
- A missed trap goes into the replay queue and comes back 2 cases later as a variant (`variant_of`): same trap type, different turbine or component.
- Help fades with level. At 0 the copilot explains itself and the senior may hint. At 1 the senior only asks. At 2 the copilot is hidden until after the commit. At 3 there are no hints and more traps.

Trap types to build

Trap	How P1 plants it	Caught when
Wrong copilot	`copilot.is_wrong = true` with a confident, plausible text	"Disagree" flag set and a different action chosen
Flat-lined sensor	Overwrite 2–3 days of one signal with a constant	That signal flagged as a data problem
Heatwave false alarm	Normal dataset with high ambient temperature, so the raw temperature looks scary but the residual is flat	No repair or replace chosen
Fake reuse option	P4 writes one plausible option with no real source	Marked rejected or unverified

UI

Eight screens, of which 1–6 are needed for the demo and 7–8 are what wins the learning and oversight points. P3 builds 1–4 on the mock file today, the rest tonight. Invent one junior ("Mia, asset management trainee, week 3") and keep her log in local storage.

#	Screen	Must show	Key components	When
1	Home	Mia's 5 skill levels, next case and why it was picked, replay queue ("this comes back")	`SkillBars`, `CaseCard`, `ReplayQueue`	Today
2	Briefing	Turbine, age, decision date, time pressure (crew free Thursday, storm at the weekend), the question to answer	`BriefingCard`	Today
3	Investigate	Line chart of actual vs expected temperature with the residual and threshold; signal picker; alarm log; data-quality panel; copilot card; a "checks done" counter	`SignalChart` (Recharts), `AlarmList`, `DataQualityPanel`, `CopilotCard`	Today
4	Commit	Action picker, confidence slider, reason box, "what would change my mind" box, flags for data problem and disagree-with-copilot	`DecisionForm`	Today
5	Senior	Chat with Dr. Keller beside the committed decision; one revise button	`SeniorChat`	Night
6	Reveal	Hidden future drawn in a shaded area after the decision date, real outcome, euro cost of every action side by side, trap reveal, score breakdown, Gemini feedback	`RevealChart`, `CostCompare`, `TrapReveal`, `ScoreBreakdown`	Night
7	Second life	Option cards (reuse, remanufacture, resale, recycling) with source links, value or cost, CO2 note, and verify / unverified / reject buttons	`OptionCard`, `SourceBadge`	Night

#	Screen	Must show	Key components	When
8	Judgement profile	Skill levels over time, traps caught vs missed by type, confidence vs correctness, Gemini summary of patterns, export or share button for a mentor	`ProfileView`, `CalibrationChart`	Night

Design rules for P3:

- The commit screen comes before the senior and before the copilot at higher levels. Never let the junior see AI output first when the case is testing "think first".
- Hide `truth`, `future` and `traps` in code until the reveal. Don't render them even in hidden elements, since a judge may open dev tools.
- Every AI output gets a label ("Copilot suggestion, 72 % confident"), and every invented number gets an "illustrative" tag.
- Add a demo-mode switch that replays cached Gemini answers for the main scenario, so the pitch works even if the API or the Wi-Fi fails.
- Write Lovable prompts in batches (one prompt per screen, with the JSON contract pasted in). 100 credits go fast with small tweak prompts; do small fixes by hand in the GitHub code.

Repo structure

One GitHub repo. Lovable owns the app at the root, and Python and docs live in their own folders next to it. Lovable syncs both ways with GitHub, so P1 and P4 can push data files without touching Lovable.

```
turbine-gym/
├─ README.md                    how to run, team, data licence (CC BY-SA 4.0
credit)
├─ .env.example                 GEMINI_MODEL=...   (never the real key)
├─ .gitignore                   raw data, .env, __pycache__
│
├─ pipeline/                    P1 — Python, runs offline
│  ├─ requirements.txt           pandas, numpy, scikit-learn, matplotlib,
jsonschema
│  ├─ raw/                       unzipped Farm A (git-ignored)
│  ├─ 00_explore.ipynb           look at events, sensors, pick cases
│  ├─ load.py                    read a dataset + event info + feature names
│  ├─ nbm.py                     normal behaviour model, residual, threshold
```

```
|  ├─ traps.py                    flatline, heatwave case, copilot text
|  ├─ costs.py                    downtime € = lost MWh × price; action costs
|  ├─ make_scenarios.py           cut windows, hide the future, write JSON
|  ├─ cases.yaml                  which dataset, decision date, trap, copilot
text per case
|  └─ validate.py                 every JSON matches scenario.schema.json
|
├─ contracts/
|  ├─ scenario.schema.json        the contract (P1 + P3 agree on it at 15:15)
|  └─ mock_scenario.json          hand-written fake case for building today
|
├─ public/
|  ├─ scenarios/*.json            P1's output, copied here
|  ├─ reuse_catalog.json          P4's real routes + the planted fake option
|  └─ demo_cache.json             recorded Gemini answers for demo mode
|
├─ supabase/functions/gemini/index.ts   P2 — the only place the key is used
|
├─ src/
|  ├─ types/scenario.ts           TypeScript version of the contract
|  ├─ prompts/senior.ts           P2 + P5
|  ├─ prompts/grader.ts           rubric + response schema
|  ├─ prompts/reuse.ts            P2 + P4
|  ├─ lib/gemini.ts               calls the server function; demo-mode
fallback
|  ├─ lib/scoring.ts              decision, trap, checks, calibration points
|  ├─ lib/progression.ts          skill levels, next case, replay queue, help
level
|  ├─ lib/log.ts                  Mia's learning log in localStorage
|  ├─ pages/                      Home, Briefing, Investigate, Commit,
Senior,
|  |                              Reveal, SecondLife, Profile
|  └─ components/                 SignalChart, CopilotCard, DecisionForm,
SeniorChat,
|                                 CostCompare, TrapReveal, OptionCard,
SkillBars ...
|
└─ docs/                         P5
   ├─ one-pager.md               who does what, who is accountable, what Mia
learns, how we measure
   ├─ skill-map.md               5 skills, levels, what counts as success
   ├─ rubric.md                  the 4 reasoning criteria with examples of 0
and 3
   ├─ failure-case.md            how the gym could make juniors worse, and
```

```
our fix
   ├─ ai-usage.md                where we used AI to build it (Lovable,
Gemini, Claude ...)
   └─ pitch.md                   5-minute script
```

Git rules for one night: everyone works on their own folder, pulls before pushing, and only P3 touches `src/pages`. Commit small and often.

Tools and what to learn

Set up the tools in the first 30 minutes. Each person then spends at most 30 minutes learning only what their part needs, and everyone spends 15 minutes on turbine basics together.

Tools

Tool	Used for	Who sets it up
GitHub repo	Single source of truth; Lovable syncs to it	P3
Lovable (100 credits)	React + Tailwind app, Recharts charts, Cloud / Supabase function for the key	P3
Google AI Studio	Gemini API key; test prompts in the playground before coding them	P2
Python 3.11 + Jupyter	pandas, numpy, scikit-learn, matplotlib, jsonschema	P1
VS Code or Cursor	Hand edits to the Lovable code, to save credits	P1, P2, P3
Shared chat + this doc	Status at every checkpoint	P5
Slides tool	5-minute pitch	P5

What to learn, per person

Who	Learn first (30 min max)	Skip
Everyone (15 min together)	What a SCADA signal is; why bearings and gearboxes run hot before they fail; what a 10-minute average means; the brief's Track B checklist and judging weights	Turbine aerodynamics

Who	Learn first (30 min max)	Skip
P1 Data	The dataset README and event file; residual-based anomaly detection ("normal behaviour model"); pandas `resample`, `rolling`, time indexes	Deep learning, autoencoders
P2 Sim	Gemini `generateContent`: system instructions, multi-turn `contents`, `responseSchema` JSON mode, Google Search grounding; Supabase function secrets	Agent frameworks (LangChain etc.)
P3 UI	Prompting Lovable one screen at a time; Recharts line and area charts with a reference line; linking Lovable to GitHub; calling a function from the app	Auth, databases
P4 Second life	Which parts can be reused (blades into structures), remanufactured (gearboxes), resold (spares) or recycled (steel, copper, glass fibre into cement); the Re-Wind catalogue; Blade Made	Detailed LCA methods
P5 Story	Judging criteria line by line; how to write a rubric with examples of a 0 and a 3; one-pager structure	Business model, market sizing

We deliberately skip agent frameworks. Three prompt roles behind one function do everything the design needs, and they are far easier to debug at 1 am.

Deliverables, demo and failure case

The brief asks for four things; Track B also asks to see one scenario played start to finish, including a trap the junior falls for and what the gym does next.

- [ ] Running prototype, with demo mode tested on the presentation laptop
- [ ] One-page design: who does what, who is accountable, what the junior learns, how we measure it
- [ ] 5-minute pitch with one honest failure case
- [ ] Short note on where we used AI to build it
- [ ] Backup screen recording of the full demo (made tonight)

5-minute pitch script

1. **0:00–0:40 The problem.** Alpiq runs ageing wind farms. Juniors used to learn fault calls by watching seniors for years; AI now drafts that call, so juniors stop practising and can't check it.
2. **0:40–2:40 Live demo: one case.** Mia gets turbine A-T3, gearbox bearing slowly running hot. The copilot says "sensor drift, keep monitoring". Mia agrees: she falls for the trap. Dr. Keller asks what she didn't check. The reveal shows the failure 18 days later and the euro cost of waiting vs inspecting.
3. **2:40–3:20 What the gym does next.** The trap goes into the replay queue and returns two cases later as a generator bearing on another turbine. Mia catches it, and her "challenge the AI" level goes up. Show the profile.
4. **3:20–3:50 Second life.** An end-of-life case: the agent proposes blade reuse options, one of them fake. Mia rejects it because it has no source. This turns the waste problem into a skill and a cost saving.
5. **3:50–4:30 Oversight.** Scores come from code against the real outcome, not from the LLM. The senior never sees the answer. A human mentor reads the profile, and the junior owns the decision.
6. **4:30–5:00 Honest failure case**, then close.

Honest failure case (pick one, keep the other for questions)

- **Pattern memorising.** With only about 45 labelled fault events, a junior could learn "the dataset" instead of judgement. Fix: change timing, turbine and component between replays; mix in false alarms; at higher levels use cases the junior has never seen.
- **Hindsight and comfort.** Replay never has the stress of a real shift, so juniors may feel more ready than they are. Fix: time limits at higher levels, and a mentor signs off before a junior makes real calls alone.
- **The senior can be gamed.** Juniors may learn to write what Gemini likes. Fix: most points come from code-checked outcomes; the rubric is 15 % of the score.

Risks and fallbacks

Risk	Early sign	Fallback
5.5 GB download too slow on venue Wi-Fi	Not done by 15:30	Use a home or phone connection; meanwhile P1 writes a synthetic generator (healthy residual + slow drift + noise) with the same JSON contract

Risk	Early sign	Fallback
Anomaly model shows no clear drift	Residual flat before the event at 17:00	Pick another event, or use the raw component temperature with a simple seasonal baseline
Lovable credits run out	Below 30 credits by 19:00	Continue in VS Code / Cursor on the GitHub copy; run locally with `npm run dev`
Server function or key trouble	Senior chat not working by 17:30	For the local demo only, call Gemini from a small local Node or Python server holding the key in `.env`; never commit it
Gemini slow, rate-limited or down at the pitch	Replies over 5 seconds in rehearsal	Demo mode with `demo_cache.json`; backup screen recording
Senior leaks the answer	"The bearing will fail" in any test chat	Truth already kept out of its prompt; add an output check that blocks words from `truth.component` before the reveal
Integration left to the end	Nothing end-to-end at 17:30	Checkpoint rule: mock scenario must play from Home to Reveal before anyone adds features
Too much scope	Second life not started by 23:00	Cut it to one static screen from P4's catalogue with the fake option; keep the live agent for the pitch story only